

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

I ATHAVAN K(192524036) that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan K

Annexure A

QUESTIONS

1. Given the three-address code sequence:

```
t1 = x * 1 t2 = t1 + y t3
= t2 - 0 t4 = t3 * 0 if t4
!= 0 goto L1 t5 = y + y
```

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2. Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

3. Construct the **DAG** for the following basic block:

```
p = a + b q = a
+ b r = p * q s
= a + b t = r + s
```

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code generator** for the code:

```
t1 = a + b t2 = c
* d
t3 = t1 - t2 t4 = e
+ f t5 = t3 / t4
```

Show the **register descriptor** and **address descriptor** after each instruction is processed.

5. For a **RISC-style target machine** with instructions **LOAD, STORE, ADD, SUB, MUL, MOV**, simulate target code generation for the expression:

$(a - b) * (c + d) - e$

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation. Given the three-address code sequence:

```
t1 = a + b,
t2 = t1 + 0, t3 = t2 * 1, if
t3 == 0 goto L1, t4 = t3
+ t3
```

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Question 1 — Peephole Optimization

Given three-address code:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Step-by-step Simulation

Original Statement	Peephole Transformation Applied	Result
t1 = x * 1	Algebraic simplification: multiplication by 1 is redundant ($x * 1 = x$).	t1 = x
t2 = t1 + y	Copy propagation: replace t1 with x (from previous step).	t2 = x + y
t3 = t2 - 0	Algebraic simplification: subtracting 0 leaves the value unchanged.	t3 = t2 $\rightarrow$ t3 = x + y (after copy propagation)
t4 = t3 * 0	Algebraic simplification / constant folding: anything multiplied by 0 is the constant 0.	t4 = 0
if t4 != 0 goto L1	Constant folding on the condition: since t4 is now the constant 0, "0 != 0" evaluates to FALSE at compile time — the branch can never be taken.	Statement removed (unreachable branch — dead code)
t5 = y + y	No redundant operator or reducible pattern in this window; expression already minimal (could optionally be strength-reduced to a shift/mult, but no benefit here).	t5 = y + y (unchanged)

Dead-Code Elimination Pass

Once the branch on t4 is removed, t4 is never used again, so its defining statement ($t4 = t3 * 0$) is dead and can also be deleted. Following the def-use chain backward:

- t4 was used only by the eliminated branch $\rightarrow t4 = t3 * 0$ is now dead $\rightarrow$ remove.
- t3 was used only to compute t4 $\rightarrow t3 = t2$ is now dead $\rightarrow$ remove (assuming t3 has no other later use).
- t2 was used only to compute t3 $\rightarrow t2 = x + y$ becomes dead in turn (again, assuming no other use).
- t1 was used only to compute t2 $\rightarrow t1 = x$ becomes dead as well.

This cascading removal is exactly how a peephole optimizer combined with dead-code elimination collapses a long dependent chain once its final consumer (the branch) disappears.

Final Optimized Instruction Sequence

```
t5 = y + y
```

If, instead, t1–t4 are known to be used later outside this window (e.g., live-out of the block), dead-code elimination cannot remove them and the safest fully-justified optimized form is:

```
t1 = x
t2 = x + y
t3 = x + y ; (t3 = t2)
t4 = 0
t5 = y + y
```

with the conditional branch removed in both cases, since it is proven unreachable by constant folding regardless of whether the temporaries are later live.

Justification Summary

- Algebraic simplification: $x * 1 \rightarrow x$, $t2 - 0 \rightarrow t2$, and recognizing $E * 0 \rightarrow 0$ remove redundant arithmetic identities.
- Constant folding: $t3 * 0$ is folded to the compile-time constant 0, and the subsequent conditional test on that constant is folded to "always false".
- Dead code elimination: the never-taken branch is deleted, and any temporary whose only consumer was that branch (or a chain leading only to it) is deleted as well, shrinking six instructions down to one.

Question 2 — Basic Blocks and Control Flow Graph

Given intermediate code:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

Rules Used to Identify Leader Statements

Rule #	Rule	Statements Identified
Rule 1	The first statement of the program/procedure is a leader.	Statement (1)
Rule 2	Any statement that is the target of a conditional or unconditional goto is a leader.	Statement (6) — target of "goto 6"; Statement (7) — target of "goto 7"
Rule 3	Any statement that immediately follows a conditional or unconditional goto statement is a leader.	Statement (4) — follows the if-goto at (3); Statement (6) — follows "goto 7" at (5) (already a leader by Rule 2)

Applying all three rules gives the complete leader set: {1, 4, 6, 7}.

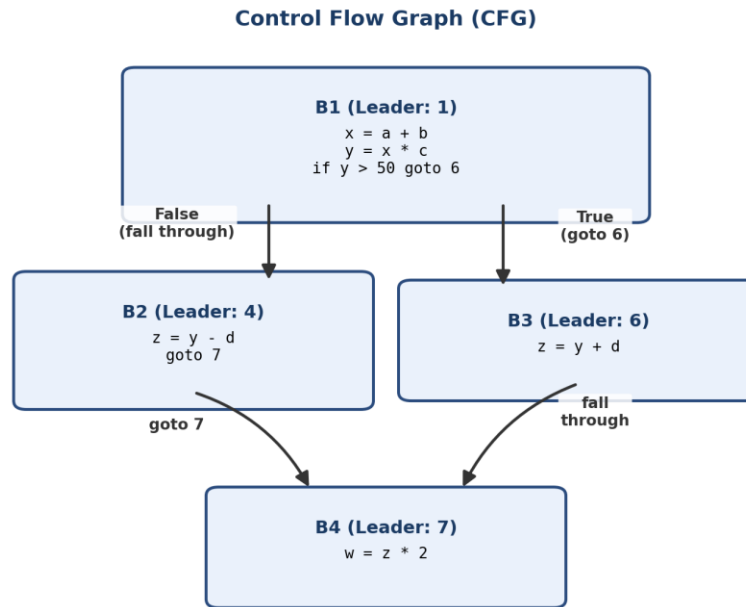
Basic Block Partition

Block	Statements	Contents	Ends With
B1	1 – 3	x = a + b; y = x * c; if y > 50 goto 6	Conditional branch
B2	4 – 5	z = y - d; goto 7	Unconditional branch
B3	6	z = y + d	Falls through
B4	7	w = z * 2	End of program

Control Flow Graph

A basic block ends with either a branch or the statement right before a leader; edges below capture the true/false outcome of the conditional in B1 and the unconditional jump in B2:

- B1 → B3 : taken when $(y > 50)$ is TRUE (goto 6).
- B1 → B2 : taken when $(y > 50)$ is FALSE (fall-through to statement 4).
- B2 → B4 : unconditional (goto 7).
- B3 → B4 : fall-through to statement 7.



Question 3 — DAG Construction and Code Generation

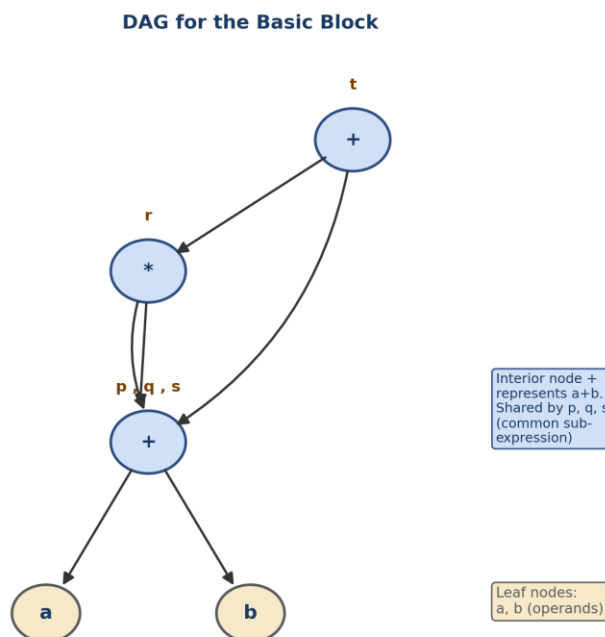
Given basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

Constructing the DAG

Processing statements in order and checking, for each new node, whether an identical operation on identical operands already exists in the DAG:

- a, b: created as leaf nodes (no incoming operations).
- $p = a + b$: no existing '+' node over (a, b) $\rightarrow$ create interior node $N1 = (+, a, b)$, attach label p.
- $q = a + b$: an identical node $N1$ already computes $a + b \rightarrow$ attach label q to $N1$ instead of creating a new node (common sub-expression detected).
- $r = p * q$: p and q both resolve to $N1 \rightarrow$ create $N2 = (*, N1, N1)$, attach label r.
- $s = a + b$: again identical to $N1 \rightarrow$ attach label s to $N1$ (second reuse, no new computation).
- $t = r + s$: r resolves to $N2$, s resolves to $N1 \rightarrow$ create $N3 = (+, N2, N1)$, attach label t.



Common Sub-expressions Identified

p, q and s all evaluate the identical expression $a + b$ and therefore collapse onto the single DAG node N1. This means the compiler only needs to compute $a + b$ once instead of three times — two redundant additions are eliminated.

Simulated Code Generation (2-register machine, R1 & R2)

Code is generated by a post-order traversal of the DAG, computing each node once and reusing its value for every label attached to it:

Step	Target Instruction	Effect / Register Allocation Decision
1	LOAD a, R1	Load a into R1 — first free register chosen since a has no register yet.
2	ADD b, R1	$R1 = a + b$, computing node N1 once. R1 now holds the value for p, q AND s simultaneously.
3	MOV R1, R2	Value of N1 is copied to R2 to preserve it for 's' (needed later in step $t = r + s$), since R1 is about to be overwritten by r.
4	MUL R1, R1	$R1 = R1 * R1 = p * q = r$ (both operands are the same node N1, so squaring R1 in place is safe and register-efficient).
5	ADD R2, R1	$R1 = r + s = t$. R2 (holding s) is added into R1, and R1 now holds the final result t.
6	STORE R1, t	Result t is stored back to memory / its home location; R1, R2 are now free for the next block.

Register allocation rationale: because p, q and s are all the same DAG node, only one register (R1) is ever needed to hold that value initially; a second register (R2) is used purely to keep a copy alive across the computation of r, since after r is computed the original $a+b$ value would otherwise be overwritten. This is the minimum possible register usage (2 registers) for this DAG.

Question 4 — Simple Code Generator with Register & Address Descriptors

Target machine has two registers R0 and R1. Given three-address code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

getReg Algorithm (Rules Applied)

- If operand y is already in a register that holds no other value and y is not needed after this instruction, reuse that register for the result.
- Else use an empty register if one is free.
- Else if the result variable has a register already assigned, reuse it.
- Else spill: pick a register to free (store its value to memory) and reuse it.

Generated Target Code

```
MOV a, R0      ; R0 = a
ADD b, R0      ; R0 = a + b = t1
MOV c, R1      ; R1 = c
MUL d, R1      ; R1 = c * d = t2
SUB R1, R0     ; R0 = t1 - t2 = t3 (t1, t2 both die here)
MOV e, R1      ; R1 = e (R1 now free, reused)
ADD f, R1      ; R1 = e + f = t4
DIV R1, R0     ; R0 = t3 / t4 = t5 (t3, t4 both die here)
MOV R0, t5     ; store final result to memory
```

Register Descriptor and Address Descriptor After Each Instruction

After Instr.	R0 holds	R1 holds	Address Descriptor (variable → location)
MOV a,R0 / ADD b,R0	t1	— (empty)	t1 → R0
MOV c,R1 / MUL d,R1	t1	t2	t1 → R0 ; t2 → R1
SUB R1,R0	t3	— (freed: t2 dead)	t3 → R0 (t1, t2 no longer needed)

After Instr.	R0 holds	R1 holds	Address Descriptor (variable → location)
MOV e,R1 / ADD f,R1	t3	t4	t3 → R0 ; t4 → R1
DIV R1,R0	t5	— (freed: t4 dead)	t5 → R0 (t3, t4 no longer needed)
MOV R0,t5	t5 (also in memory)	—	t5 → R0 and memory

Because t1 and t2 are each used exactly once (by the very next instruction) and are not needed afterward, the code generator reclaims R0 and R1 immediately after SUB and DIV rather than allocating new registers — this is exactly rule 1 of getReg, and it lets the entire five-statement block execute using only two registers with zero spills.

Question 5 — RISC Target Code Generation and Runtime Storage

Expression to compile: $(a - b) * (c + d) - e$, for a RISC machine with LOAD, STORE, ADD, SUB, MUL, MOV, where all arithmetic operands must first be brought into registers (load/store architecture).

Three-Address Code

```
t1 = a - b
t2 = c + d
t3 = t1 * t2
t4 = t3 - e
```

Register-Minimal Evaluation Order

Using a Sethi–Ullman style ordering, sub-expressions are evaluated so registers are freed and reused as soon as an operand is no longer needed, keeping the register count to just 2 for this expression.

Generated RISC Target Code

```
LOAD R1, a      ; R1 = a
LOAD R2, b      ; R2 = b
SUB  R1, R1, R2  ; R1 = a - b      (t1; R2 now free)
LOAD R2, c      ; R2 = c          (reuse freed R2)
LOAD R3, d      ; R3 = d
ADD  R2, R2, R3  ; R2 = c + d      (t2; R3 now free)
MUL  R1, R1, R2  ; R1 = (a-b) * (c+d) (t3; R2 now free)
LOAD R2, e      ; R2 = e          (reuse freed R2)
SUB  R1, R1, R2  ; R1 = t3 - e = result (t4)
STORE result, R1 ; write final value to its home location
```

Only 3 registers are needed at peak (R1–R3), and R2/R3 are recycled twice each — no spilling to memory is required for an expression of this size.

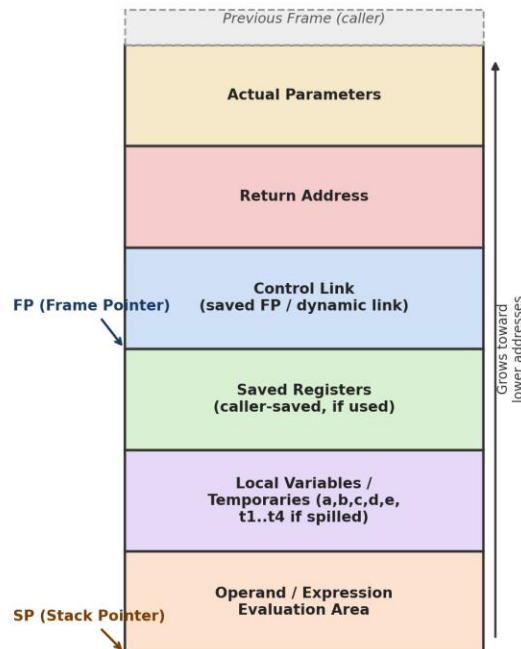
Runtime Storage Management and Stack Frame Layout

If this expression evaluation occurs inside a called procedure, the runtime system uses a stack-based activation record (stack frame) for the procedure, allocated by the caller/callee prologue and released in the epilogue:

- Actual parameters — values/addresses passed in by the caller, placed at a fixed offset from the frame pointer.
- Return address — where control resumes in the caller after the procedure completes; pushed automatically by the CALL instruction.

- Control link (dynamic link) — saved value of the caller's frame pointer, restored on return so the caller's frame becomes active again.
- Saved registers — any caller-saved registers (here, R1–R3 if they held live caller values) are pushed before use and popped before returning.
- Local variables / temporaries — storage for a, b, c, d, e if they are locals, and for spill slots of t1–t4 if the register set were too small.
- Operand/evaluation area — extra temporary stack space some RISC ABIs reserve for evaluating deeply nested expressions that exceed the available registers.

Activation Record / Stack Frame Layout



The Frame Pointer (FP) stays fixed for the duration of the procedure and is used to address parameters and locals with constant offsets, while the Stack Pointer (SP) moves as temporaries are pushed/popped during expression evaluation. On procedure entry, SP is decremented to reserve the frame; on exit, the control link restores the caller's FP, the return address is popped into the program counter, and SP is restored to its pre-call value — safely tearing down the frame built for evaluating $(a - b) * (c + d) - e$.